Lab class 5 Exercise Sheet

Model answers

Question 1:

You want to lay out some domino pieces in a single line such that adjacent pieces show the same number on the halves facing each other. After some failed attempts, you start wondering if this is even possible with the pieces that you have. With your network science hat on, how do you go about answering this question?

To measure the sum of all pairwise distances between all stakes, we need to string the rope in such a pattern that it traces all pairwise distances once. Let's consider the graph where the stakes are nodes and a link indicates that the rope connects the respective pair of stakes. To measure the sum of the pairwise distances, we need to create a fully connected graph. We can trace a given path with a single rope if the graph has an *Eulerian path* (see 3rd lecture). We know from Euler's bridge theorem that such a path exists if there are at most two nodes of odd degree. In a fully connected network, each node has the same degree, and the degrees are odd when the number of nodes is even, and vice versa. So the answer is, we can measure the sum of all pairwise distances between stakes if either the number of stakes is less than three or if it is odd.

Question 2: About small-world networks

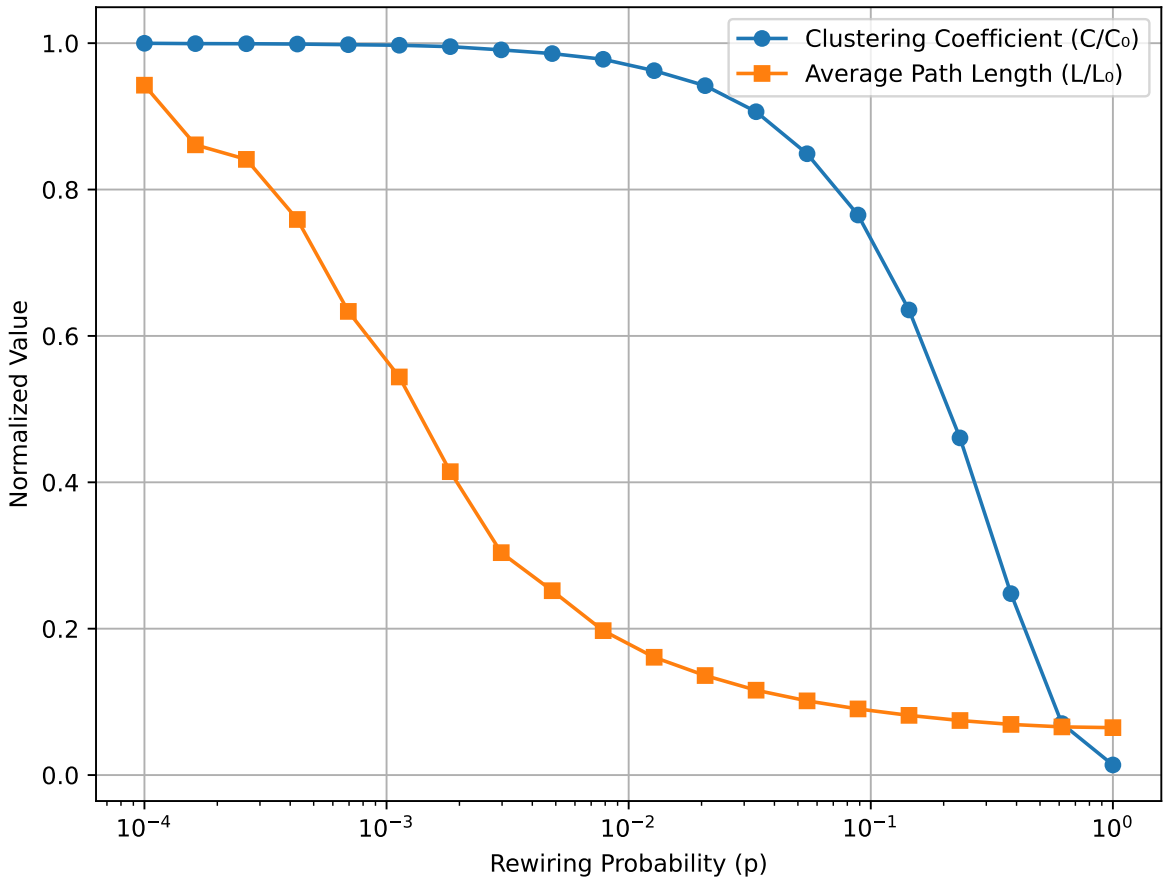
Write the Python code to reproduce the Watts-Strogatz small-world model [1], i.e., starting from a regular network (decide degree), and for a range of probabilities p from 0.00 to 1.00 (in steps of 0.0001 for example), rewire the network with probability p. When you write the rewiring instruction, considers the possible scenarios. Plot the figure showing the evolution of the average path length, the global clustering coefficient and the small-worldedness index with p.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
import random
```

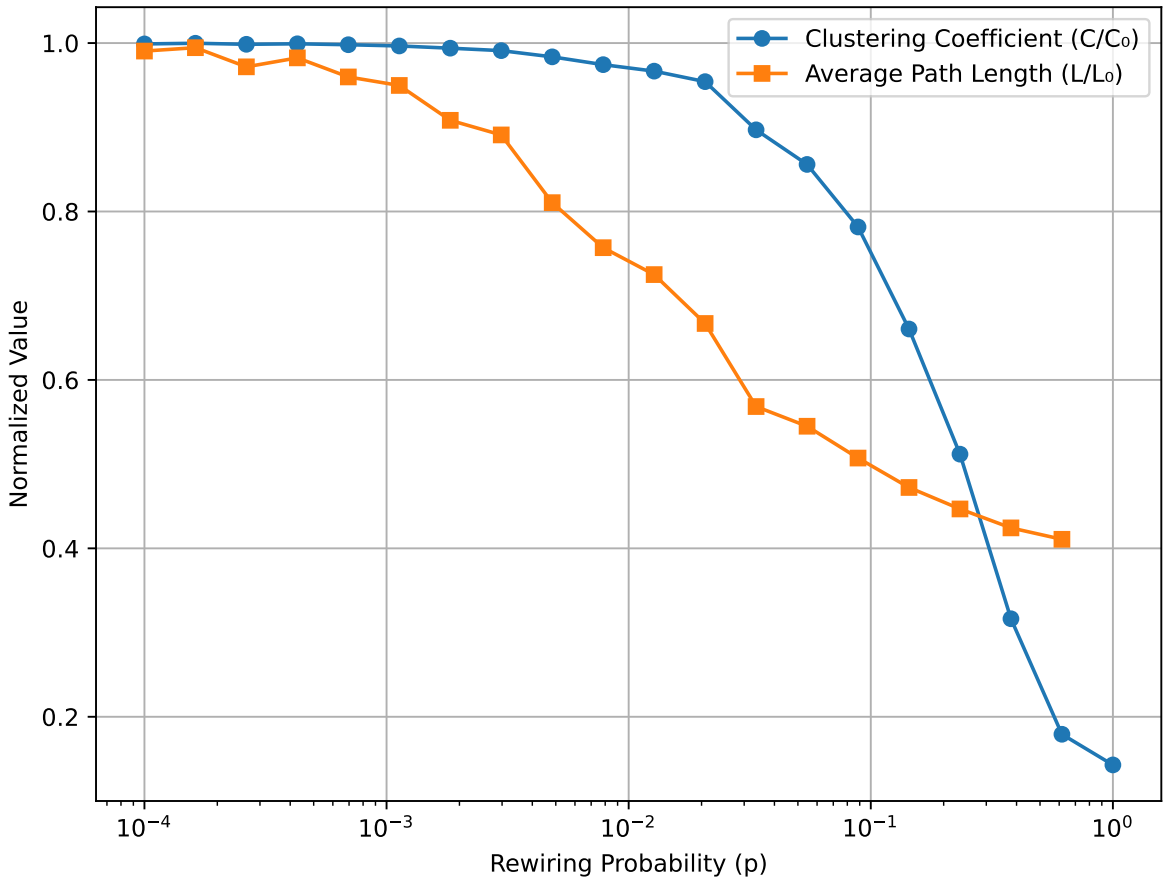
```
def rewire_ws_style(G, p):
    """Rewire edges in Watts-Strogatz fashion: edges are rewired randomly
    without enforcing degree constraints."""
    new_G = G.copy()
    edges = list(G.edges())
    nodes = list(G.nodes())

    for u, v in edges:
        if random.random() < p:  # Rewire with probability p
            new_G.remove_edge(u, v)

            # Pick a completely random node (Watts-Strogatz does NOT
            # preserve degrees strictly)
            new_v = random.choice(nodes)
            while new_v == u or new_G.has_edge(u, new_v):  # Avoid self-loops
              #and duplicate edges
                new_v = random.choice(nodes)

            new_G.add_edge(u, new_v)

    return new_G

# Parameters
N = 1000  # Number of nodes
k = 10    # Each node connects to k nearest neighbors
rewiring_probs = np.logspace(-4, 0, 20)  # Log-spaced values from 0.0001 to 1
num_repetitions = 20  # Number of repetitions for averaging

# Create the initial k-regular ring lattice once
# You could use regular_random_graph as well.
initial_ring_lattice = nx.Graph()
for i in range(N):
    for j in range(1, k // 2 + 1):  # each node has k/2 neighbors either side
        initial_ring_lattice.add_edge(i, (i + j) % N)
        initial_ring_lattice.add_edge(i, (i - j) % N)

# Compute L_0 and C_0 for the lattice.
# This is to normalise the curves so that for $p=0$, both clustering
# coefficient and shortest path length has value 1.
L_0 = nx.average_shortest_path_length(initial_ring_lattice)
C_0 = nx.average_clustering(initial_ring_lattice)
```

```
# Storage for results
clustering_coeffs_ws = np.zeros(len(rewiring_probs))
path_lengths_ws = np.zeros(len(rewiring_probs))

# Perform averaging over multiple runs
for _ in range(num_repetitions):
    clustering_coeffs = []
    path_lengths = []

    # Compute metrics for different p values
    for p in rewiring_probs:
        G = rewire_ws_style(initial_ring_lattice, p)  # WS-style rewiring

        # Compute clustering coefficient
        C = nx.average_clustering(G)

        # Compute shortest path length (only if the graph is connected)
        if nx.is_connected(G):
            L = nx.average_shortest_path_length(G)
        else:
            L = np.nan  # Handle disconnected graphs (possible for large p)

        clustering_coeffs.append(C / C_0)  # Normalize by C_0
        path_lengths.append(L / L_0)  # Normalize by L_0

    # Accumulate for averaging
    clustering_coeffs_ws += np.array(clustering_coeffs)
    path_lengths_ws += np.array(path_lengths)

# Compute final averages over normalized values
clustering_coeffs_ws /= num_repetitions
path_lengths_ws /= num_repetitions

# Plot results
plt.figure(figsize=(8, 6))

plt.plot(rewiring_probs, clustering_coeffs_ws, 'o-',
  label="Clustering Coefficient (C/C )")
plt.plot(rewiring_probs, path_lengths_ws, 's-',
  label="Average Path Length (L/L )")

plt.xscale("log")
```

```
plt.xlabel("Rewiring Probability (p)")
plt.ylabel("Normalized Value")
plt.legend()
plt.grid(True)
plt.show()
```



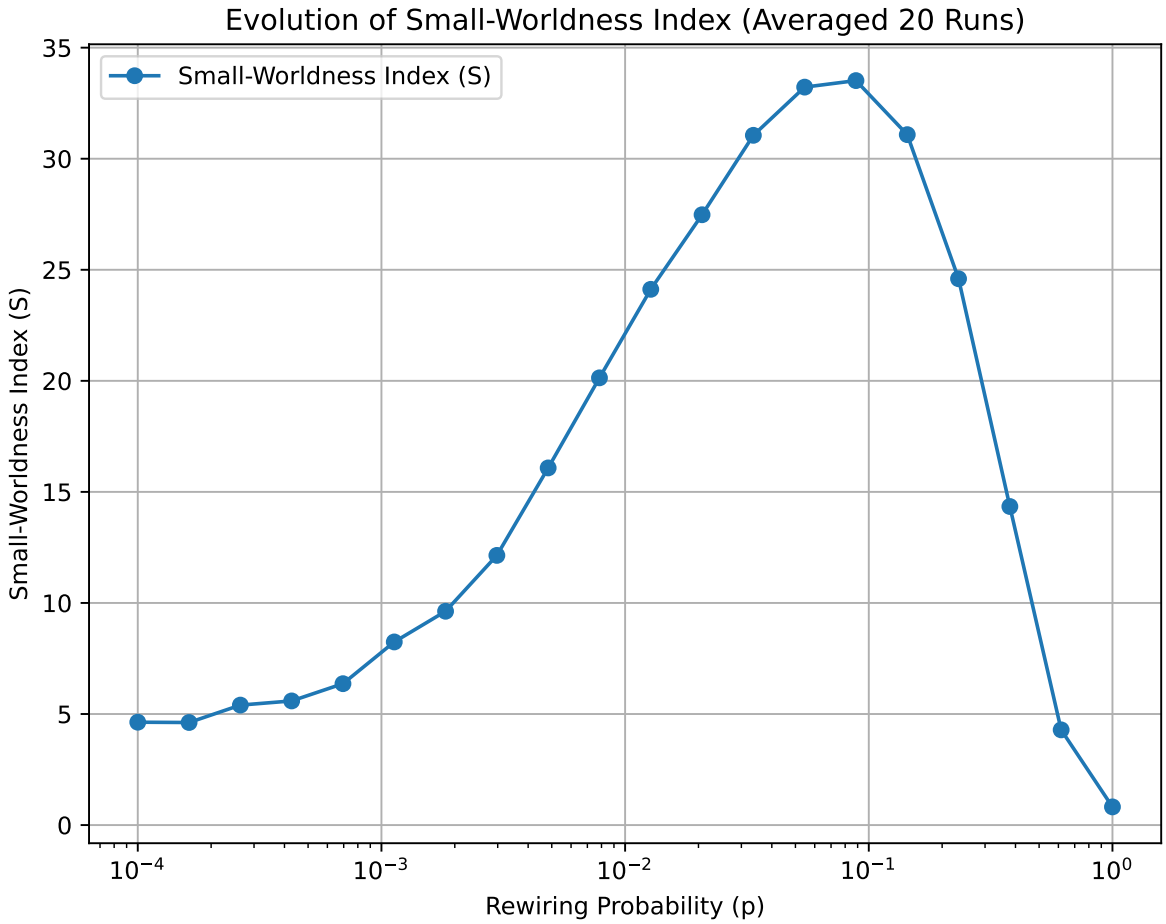
In the code above, we avoid self-loops and duplicate edges. Note that we do not try to preserve the degree sequence. An implication of that is that potentially the network could become disconnected. Watts and Strogatz used $N = 1000$ nodes. What happens if you use the code above but now use $N = 100$?



You should notice that the normalised APL is more around 0.4 than < 0.2 as in the previous figure. Why? It is actually possible to predict what that value should be. In a random network, the APL is expected to be $\approx \frac{\log N}{\log\langle k\rangle}$, which, in the case of $N = 100$ and $k = 10$ is 2. Now, for the regular ring lattice, we can estimate the APL as follows. The APL is going to be the average distance between nodes divided by $k/2$ (since from each node, we can jump straight to a node $k/2$ edges away – that's the fastest way to go around the ring). What is the average distance between nodes? You could calculate it but it's a bit unpleasant. Instead, we can approximate it. We say that the nodes are uniformly distributed around the ring and therefore the distances follow a triangular distribution between 0 and $N/2$? Why $N/2$? because on a ring, the longest 'distance' is $N/2$. The average of such distribution is $N/4$. Therefore, the estimated APL is $(N/4)/(k/2) = N/(2k)$. So for $N = 100$ and $k = 10$, we estimate $L_0 = 5$. And therefore, the normalised APL for $p = 1$ (when the network is random) is $\approx 2/5 = 0.4$. Now, when $N = 1000$ (for the same k), we have $L_0 \approx 50$ and the expected APL for $p = 1$ (when the network is random) is 3 and now the normalised APL is $\approx 3/50 = 0.06$ which is more in line with what we observe with $N = 1000$!

In fact, we can use this information to quickly calculate the small-worldness index rather

than simulating a large number of ER equivalent networks. Namely, we will use the fact that $C_{ER} \approx k/N$ and $L_{ER} \approx \log(N)/\log(k)$ and we get:



Small-worldness index is close to 1 for $p = 1$ which is expected.

Question 3: About ERGMs (a bit advanced)

In a previous lab class, we looked at generating networks with low or high assortativity but (a) we didn't specify a target value and (b) we didn't control for other properties. That is something we can do by using ERGMs. So, in this exercise, you are going to download some dataset (pick whatever you like but avoid something too big or too small) and then decide 2 or 3 properties you might want to preserve, e.g., average degree, a global clustering coefficient and degree assortativity. [long explanation snipped] Using a MCMC sampler, generate a number of samples and compare their network statistics with those of the original network.

In what follows I will use the classical Karate club graph (which you can load straight from networkx). Its average degree $\langle k \rangle = 4.72$, degree assortativity $r = -0.48$ and clustering $c = 0.26$. First, we need to infer the parameter θ given the model. Once we have estimated the parameters, we can sample from the distribution of graphs with those parameters. Note that the sampler is used both for inference and for generation.

> ⚠ Warning
>
> Monte-Carlo based simulations are very long so you may not actually want to launch this code. On my computer, it took a good few hours to return the estimated parameters.

Once you have the estimated parameters, you can generate 'samples' as follows:

This process is not too long so you can try. The code uses a burn-in period of 1000 steps. This is probably too short to get good results even for such small networks.

> ℹ Note
>
> There is a Python package for implementing ERGMs called pyERGM but the documentation is somewhat incomplete. Network metrics can be implemented but require some attention. This is actually a spin-off of a R package for ERGM which is worth looking at if you are comfortable with R. You can find it here

References

1. D. J. Watts & S. H. Strogatz, Collective dynamics of "small-world" networks. *Nature*, **393** (1998) 440–442. https://doi.org/10.1038/30918.